

The Limitless Stack

A complete AI operating protocol — clone, install, run.

Seven tools wired into one operating system: **Claude**, **CLAUDE.md**, **Obsidian**, **NotebookLM**, **Pinecone**, **Hub Workspace**, **Paperclip**. This guide takes you from *git clone* to a working stack in roughly 30–45 minutes, including six installable skills, a session-start preflight that verifies all seven tools, and an end-of-session checklist that guarantees nothing is left uncommitted, unsynced, or unverified.

Read it once before you start; paste it into your Claude session as the onboarding context so your Claude inherits the same operating discipline this stack runs on.

What you'll have when you're done

- **A working knowledge base** in Obsidian that compounds — every new source updates many pages, never derived from scratch.
- **Semantic recall** across every repo and document via Pinecone — natural-language search, no grep gymnastics.
- **Curated topic notebooks** in NotebookLM, including a special *reminder bucket* your Claude reads at the start of every session.
- **Mechanical session rituals**: a Roll Call preflight at start and an 8-step checklist at end. Both bundled as installable skills, both enforced by your CLAUDE.md.
- **A living anti-patterns log** — institutional memory of mistakes worth not repeating, ships from day one.
- **Six skills** auto-installed to `~/ .claude/skills/` by the installer. See pages 4–5 for what each one does.

The seven tools

Each tool plays one role in the larger system. None is novel on its own; the protocol is in how they're connected.

Tool	Role	What it owns
Claude	Reasoning engine	Reads, writes, connects, decides
CLAUDE.md	Identity + rules	Trust anchor, voice, guardrails per context
Obsidian	Knowledge base	Structured wiki: architecture, decisions, entities
NotebookLM	Research desk	Deep research across curated source collections
Pinecone	Semantic memory	Full-text recall across every repo and document
Hub Workspace	Multi-model agent runtime	Chat surface — Gemini default, Claude opt-in
Paperclip	Coordination	Org chart, budgets, tickets, approvals

Why this exists — capabilities you didn't have before

AI assistants have an amnesia problem: every session starts cold. Long contexts balloon costs and degrade reasoning. Classic RAG (chunks → answer → forget) doesn't compound. The Limitless Stack solves this by giving Claude a persistent memory layer it can read AND write — knowledge that gets smarter every time you use it.

Knowledge that compounds. One new source can update fifteen wiki pages. Each query is informed by every prior ingest. The wiki gets sharper over time.

Mechanical session rituals. Roll Call (preflight) verifies all seven tools are present and in sync before substantive work. The end-of-session checklist guarantees nothing is left uncommitted, unsynced, or unverified. Both bundled as installable skills.

Cross-repo semantic memory. Pinecone indexes every repo and document. Natural-language queries surface the right files in seconds.

The reminder bucket. A curated NotebookLM notebook with five files that your Claude queries FIRST every session. The single most powerful piece of session-state continuity in the stack.

Living anti-patterns log. When a session catches a recurring mistake, it lands in `wiki/synthesis/claude-anti-patterns.md`. Future sessions read it via the reminder bucket. The system gets smarter from each mistake.

The six skills — what they are, what they do

These are the heart of the stack's operational discipline. Each skill is a self-contained Markdown file Claude can read on demand. A skill's **description** field is what makes it auto-trigger — when Claude sees a phrase that matches the description, the skill loads and Claude follows its instructions.

How they get installed: running `./install.sh ~/your-vault` copies all six skills to `~/claude/skills/`. From that point on, every Claude Code or Cowork session can discover and invoke them.

1. limitless-stack — the umbrella protocol

Installed at: `~/claude/skills/limitless-stack/SKILL.md`

The canonical 400+ line protocol document. Contains the seven-tool framing, the four-tool lookup order (wiki → Pinecone → NotebookLM), the **Mandatory First Action** (5 steps), the **End-of-Session Checklist** (8 steps), the reminder bucket pattern, the anti-patterns reference, and the code-discipline rules.

Triggers when the user references the Limitless Stack, OpenScaffold work, or asks about the seven-tool protocol. Every session that touches this vault should have this skill loaded — that's why it's auto-invoked via CLAUDE.md's mandatory-first-action banner (see page 6).

2. roll-call — session-start preflight

Installed at: `~/claude/skills/roll-call/SKILL.md`

Mechanically verifies all seven tools are present, authenticated, and in sync before substantive work starts. Calls `tools/limitless-preflight.sh` via Desktop Commander, which checks: vault exists and is committed, Pinecone API key in Keychain and the index responds, NotebookLM CLI authed and notebooks exist, no duplicates across the seven buckets, the Hub workspace is reachable, Paperclip integration health.

Returns: exit 0 = READY (proceed) · exit 1 = WARN (proceed with caveats, report findings) · exit 2 = BLOCK (do NOT proceed until fixed).

Triggers when the user says "hey claude", "are you ready", "roll call", "preflight", "is everything connected" — or any greeting that opens a session. The skill description specifically tells Claude: *do not begin substantive work until Roll Call returns READY*.

3. notebooklm — full NotebookLM CLI API

Installed at: `~/claude/skills/notebooklm/SKILL.md`

565-line skill bundling the entire `notebooklm-py` CLI. Lets Claude create notebooks, add sources, query, generate artifacts (audio overviews, study guides, FAQs, briefings, mind maps), and download results — all via the CLI on your Mac, never inside an ephemeral sandbox (which would lose Google auth).

Triggers when the user references NotebookLM, asks to create a notebook, wants to add sources to an existing notebook, or asks for a podcast / overview / study guide.

The six skills (continued)

4. four-tool-lookup — the search discipline

Installed at: `~/ .claude/skills/four-tool-lookup/SKILL.md`

Codifies the lookup order — `wiki/index.md` → relevant wiki pages → Pinecone semantic search → NotebookLM (if deep research needed) → only THEN reason from context. Separated from the umbrella skill so the discipline is enforceable on its own when a session is doing pure research.

Triggers when the user asks a factual or architectural question about the vault's domain. The skill makes Claude show its work — "I checked X, then Y, then Z" — so you can see whether the lookup order was followed.

5. verify-before-claim — guards against false unavailability

Installed at: `~/ .claude/skills/verify-before-claim/SKILL.md`

Born from anti-pattern #12: Claude hits one error in one execution environment (e.g. Pinecone unreachable from a sandbox), then tells the user "Pinecone isn't available" — instead of trying the same call from Desktop Commander on the Mac where it works. Wastes time, erodes trust.

Triggers when Claude is about to declare any tool, resource, service, or capability as "unavailable", "unreachable", "not working", or "not accessible". The skill makes Claude stop, enumerate alternative execution paths, and verify the failure across all of them before claiming.

6. karpathy-guidelines — code-edit discipline

Installed at: `~/ .claude/skills/karpathy-guidelines/SKILL.md`

Mirror of `forrestchang/andrej-karpathy-skills` (MIT licensed). Four named principles distilled from Karpathy's public observations on LLM coding pitfalls:

- **1. Think Before Coding** — surface assumptions, present alternatives, ask before guessing.
- **2. Simplicity First** — minimum code that solves the problem; no speculative abstractions.
- **3. Surgical Changes** — touch only what's required; every changed line traces to the user's request; no drive-by refactors.
- **4. Goal-Driven Execution** — write multi-step plans as [Step] → verify: [check] so each step has its own success criterion.

Triggers when the user is writing, reviewing, or refactoring code. The skill biases toward caution — the tradeoff is fewer drive-by improvements but much higher signal-to-noise in PRs.

Operational tools shipped alongside the skills

`install.sh` also copies six tools into your vault's `tools/` directory: `limitless-preflight.sh` (the script Roll Call calls), `session-bootstrap.sh` (current-thesis snapshot), `notebooklm-wiki-refresh.py` (routes wiki files into the right NotebookLM buckets, verifies content actually landed), `notebooklm-dedupe.py` (sweeps any bucket for duplicates), `pinecone-sync.py` (chunks the corpus and upserts to the index), `pinecone-search.py` (semantic search). The `preflight` + `refresh` + `dedupe` scripts have hard-coded bucket IDs that you'll edit during install (see page 9).

Prerequisites

Accounts you need

- **GitHub** with access to the org whose repos you'll index. PAT with repo scope if you don't already have CLI auth.
- **Pinecone** (free Starter tier is fine).
- **Google account with NotebookLM access** — logged into Chrome.
- **Invitation to Open-Scaffold-Labs/LimitlessStack** as a collaborator.

Software on the Mac

- **Homebrew, Python 3.11** (`brew install python@3.11`), **Node.js**, **Obsidian**, **gh CLI**, **Chrome** with the Claude extension.

Step-by-step install

1. Clone and install

```
$ gh repo clone Open-Scaffold-Labs/LimitlessStack ~/LimitlessStack
$ cd ~/LimitlessStack
$ ./install.sh ~/your-vault-path
```

The installer copies all six skills to `~/ .claude/skills/`, the vault template (including four foundational pages) to your vault's `wiki/`, and operational tools to `tools/`.

Verify: `ls ~/ .claude/skills/` shows six directories. `ls ~/your-vault-path/wiki/` shows `index.md`, `log.md`, `overview.md`, plus the four foundational pages.

2. Pinecone — Keychain + first sync

```
$ security add-generic-password -a pinecone -s pinecone-api-key -U -w
# (paste your Pinecone API key when prompted)

$ cd ~/your-vault-path
$ python3.11 tools/pinecone-sync.py
```

Verify: `python3.11 tools/pinecone-search.py "any test query"` returns results.

3. Create your NotebookLM buckets

In Chrome (signed into NotebookLM), create at minimum two notebooks: a **Wiki default** and a **Reminder bucket**. Note each notebook's ID. Per-project notebooks (one per significant repo) are optional but recommended.

4. Wire your bucket IDs into the config

Edit `tools/notebooklm-wiki-refresh.py` — replace Matt's bucket IDs (`NOTEBOOK_ROUTES`, `DEFAULT_ROUTE`, `REMINDER_NOTEBOOK_ID`, `REMINDER_FILES`) with yours. Same for any bucket-ID block in `tools/limitless-preflight.sh`.

5. Customize for your domain + run Roll Call

Edit ~/your-vault-path/CLAUDE.md — replace [YOUR DOMAIN] placeholders. Then verify the whole stack:

```
$ bash tools/limitless-preflight.sh ; echo "exit: $?"
```

Exit 0 = READY · 1 = WARN · 2 = BLOCK.

How the session rituals are enforced

This is the section that answers "OK, but how do I actually make my Claude run Roll Call and the end-of-session checklist every session?" There's no single magic hook — instead, three layers of enforcement work together. Understanding all three is the difference between Claude following the protocol by accident and Claude following it reliably.

Layer 1 — CLAUDE.md's mandatory-first-action banner (the primary hook)

Your vault's CLAUDE.md (seeded by `install.sh` from the schema template) opens with this banner:

```
> ■ MANDATORY FIRST ACTION – NO EXCEPTIONS
> Before answering ANY question, making ANY file changes, or starting ANY task:
> 1. Invoke the roll-call skill.
> 2. Run tools/session-bootstrap.sh.
> 3. Query the reminder NotebookLM bucket for current operating rules.
> 4. Read wiki/index.md.
> 5. Do NOT skip these steps.
```

Claude Code and Cowork read CLAUDE.md automatically at session start. The banner is the first thing in the file, so it's the first thing Claude sees. Combined with the explicit "NO EXCEPTIONS" framing and the numbered step list, Claude treats it as a hard precondition.

What you do: nothing. The banner is in the schema template `install.sh` drops at `~/your-vault/CLAUDE.md`. Don't delete it. If you customize the file, leave the banner intact.

Layer 2 — skill descriptions auto-trigger on greetings

Each skill has a **description** field listing trigger phrases. Claude matches user messages against these and auto-invokes the skill. The `roll-call` skill description specifically lists:

```
"hey claud", "are you ready", "is everything connected",
"roll call", "preflight", "pick up where you left off",
any greeting that opens a session
```

So even if Claude somehow blew past the CLAUDE.md banner (it won't, but defense in depth), the moment you say "hey claud" or "let's pick this back up", the Roll Call skill loads and Claude follows its instructions.

Layer 3 — the bootstrap prompt as a manual backup

If you want belt-and-suspenders, paste this prompt at the start of any session where you want to be 100% sure the ritual runs:

```
Hey Claude. This vault uses the Limitless Stack protocol. Run roll-call,
then bootstrap, then query my reminder NotebookLM bucket for the current
operating rules and recent anti-patterns. Then read wiki/index.md.
Then we'll start work.
```

This is what you do when you're trying out a new model, a fresh API context, or any environment where you can't be sure CLAUDE.md will load reliably. The explicit prompt is the failsafe.

Verifying it ran: after Claude responds, you should see Roll Call output (READY/WARN/BLOCK with per-tool checks). If you don't see that, ask: "did you run Roll Call?" — if Claude didn't, ask it to do so before continuing.

End-of-session — the 8-step checklist and how to enforce it

Same three-layer enforcement model, applied to wrap-up. The difference: end-of-session has more failure modes (uncommitted wiki, unsynced Pinecone, NotebookLM refresh that silently no-op'd), so verification is load-bearing.

The 8-step checklist (lives in `CLAUDE.md` and the `limitless-stack` skill)

- 0. Update `wiki/team-tasks.md` + `wiki/my-tasks/<your-login>.md`.
- 1. Commit and push the wiki repo. Every session, no exceptions.
- 2. Push any other repos modified.
- 3. Verify `wiki/log.md` has entries for everything done.
- 4. `python3.11 tools/pinecone-sync.py --changed-only`
- 5. `python3.11 tools/notebooklm-wiki-refresh.py`
- 6. **VERIFY the refresh actually landed** — script must report `verify_failed: 0`; if not, query NotebookLM directly.
- 7. If reminder allowlist files changed, query the reminder bucket explicitly to confirm the answer reflects the edit.

Steps 6 and 7 are not optional. They exist because `notebooklm source refresh` was silently a no-op for file sources for two weeks while the script happily reported success. That's anti-pattern #12. The verify step catches it.

How it gets enforced

Layer 1 — `CLAUDE.md`. The end-of-session checklist is in your vault's `CLAUDE.md` as a labeled section. Claude reads it on session start (alongside the mandatory-first-action banner), so the rules are in context throughout the session.

Layer 2 — the `limitless-stack` skill. The skill's `SKILL.md` restates the checklist, so even sessions where `CLAUDE.md` isn't fully internalized, the skill loads when the user says "wrap up" or "end of session" and Claude has the checklist front-of-mind.

Layer 3 — explicit user prompt. The phrase that's most reliable: *"Run the end-of-session checklist."* Claude will literally walk through all eight steps, run the scripts, report `verify_failed` counts, and surface anything that didn't land.

Pro tip: the next session's Roll Call (Layer 1 of session start) catches most failures from the previous session's wrap. If Step 1 (commit + push) was skipped, Roll Call's git-status check fires WARN. If Step 6 (verify NotebookLM) was skipped, the next dedupe sanity check catches it. The two rituals are designed to validate each other.

Day-to-day operating

Ingest a new source

Drop the file into `raw/`. Then ask your Claude: *"Ingest the new source at `raw/{filename}`."* Claude reads it, creates `wiki/sources/<slug>.md`, updates entity/concept pages it touches, flags contradictions, updates `wiki/index.md`, and appends a log entry.

Ask a question

Just ask. Claude follows the four-tool lookup: `wiki/index.md` → relevant wiki pages → Pinecone semantic search → NotebookLM (if deep research needed) → only then reason from context. Substantive answers get filed back as `wiki/synthesis/<slug>.md`.

Periodic lint

Every few weeks: *"Run a lint pass and write the report."* Surfaces contradictions, stale claims, orphan pages, missing entities, broken links, coverage gaps, index drift. Report lands at `wiki/synthesis/lint-YYYY-MM-DD.md`.

Catch a new failure mode

When a session catches a recurring mistake worth not repeating, append a numbered entry to `wiki/synthesis/claude-anti-patterns.md`. Format: trigger pattern, why-it's-tempting, why-it's-wrong, the corrective rule. Future sessions read it via the reminder bucket and learn from it.

Schedule a nightly Pinecone sync

Use `crontab -e` or your scheduler of choice to run `tools/pinecone-sync.py --changed-only` overnight. Keeps the index current without ceremony.

Customization checklist — disconnect from the worked example

When you clone, you're getting Matt's setup as a worked example. To make it yours:

- **Bucket IDs** in `tools/notebooklm-wiki-refresh.py` (`NOTEBOOK_ROUTES`, `DEFAULT_ROUTE`, `REMINDER_NOTEBOOK_ID`, `REMINDER_FILES`) and any bucket-ID block in `tools/limitless-preflight.sh`.
- `CLAUDE.md` — replace `[YOUR DOMAIN]` placeholders.
- **Wiki entity links** — `[[entities/openscaffold]]` in the foundational pages can stay as worked examples or be replaced with your org page.
- **Repo clone list** in `raw/` — replace OpenScaffold-specific examples with your own repos.
- **Keep as-is:** all six skills, generic operational tools, vault directory structure.

Reference card

The six skills (one-line summaries)

Skill	What it does
<code>limitless-stack</code>	Umbrella protocol: 4-tool lookup, mandatory first action, end-of-session checklist, reminder bucket, anti-
<code>roll-call</code>	Session-start preflight. Verifies all 7 tools. Returns READY/WARN/BLOCK.
<code>notebooklm</code>	Full NotebookLM CLI API. Notebooks, sources, queries, artifacts.
<code>four-tool-lookup</code>	Wiki → Pinecone → NotebookLM lookup discipline.
<code>verify-before-claim</code>	Guards against false 'unavailable' claims. Verify across execution paths.
<code>karpthy-guidelines</code>	Code-edit discipline (4 named principles). MIT mirror of forrestchang/andrey-karpthy-skills.

Operational tools

Tool	What it does
<code>limitless-preflight.sh</code>	The script Roll Call calls. Validates 7 tools, dedupe-sweeps every NotebookLM bucket, returns 0
<code>session-bootstrap.sh</code>	Current-thesis snapshot at session start.
<code>notebooklm-wiki-refresh.py</code>	Routes wiki files into per-project + default + reminder buckets. Verifies content actually landed.
<code>notebooklm-dedupe.py</code>	Sweep any bucket for duplicates.
<code>pinecone-sync.py</code>	Chunks corpus, upserts to index. --changed-only, --dry-run, --repo NAME.
<code>pinecone-search.py</code>	Semantic search with optional --repo filter.

Foundational reading (in obsidian/vault-template/wiki/)

- `concepts/llm-wiki-pattern.md` — Karpathy's LLM Wiki pattern. The methodology this vault is built on.
- `sources/claude-code-karpathy-obsidian-video-2026-04-14.md` — Jack Roberts video that distilled the four-tool framing.
- `concepts/notebooklm-workflow.md` — The seven-bucket routing pattern.
- `synthesis/claude-anti-patterns.md` — Living log of failure modes worth not repeating. 11+ entries on day one; you'll add yours.

Where to dig deeper

- `skills/limitless-stack/SKILL.md` — full protocol, every section, 400+ lines.
- `docs/setup-guide.md` — detailed step-by-step (this PDF is the highlight reel).

- `README.md` — what's in the repo.
- Upstream: `forrestchang/andrej-karpathy-skills` for the `karpathy-guidelines` source.

The Limitless Stack is one integrated system. Don't try to use just one tool — the value compounds across all seven. Welcome aboard.